1. Introduction. A *path domino* is a 1×2 tile with a curve drawn on it. The curve enters and leaves through the midpoints of the six unit edges on the tile's boundary, so a tile is decided by a matching of those six points—no point used twice, and any number of them left alone. Two tiles are the same if one is the other turned through 180°.

Exercise 151 of §7.2.2.1 counts the tiles that use two or four of the six points—nine of the first kind and twenty-seven of the second—and asks for all thirty-six of them in an 8×9 array, so that the curves join into a single closed loop. Exercise 152 completes the set with the blank tile and the eleven that use all six points, and asks for all forty-eight in an 8×12 array. Its answer then notices that exactly thirty-two of the forty-eight never cross themselves, that thirty-two tiles cover sixty-four cells, and that a chessboard therefore invites them—but reports that the invitation cannot be accepted.

2. I wrote this program in September 2026 to check both exercises and both answers. Everything they assert about the pieces, the printed figures, the models, and the counts came out right; the notes are the companion document `README.md` in the directory above.

Two things here go past the book. The first is a repair for the chessboard: the thirty-two non-crossing tiles miss by exactly one piece, and a single closed curve that crosses itself just once does exist—so one is not only attainable but minimal. The second is the 8×12 array of exercise 152 itself, which the answer reports finding with luck and 35.1 teramems. This program found one too, and it is drawn at the end.

3. The whole method rests on one idea from the answer to 151. Solving for the tiles directly is hopeless, because a tile is a matching and matchings do not compose. So *factor* the problem: forget how the six points are joined and remember only which of them are switched on. The on/off pattern is a six-bit mask, the mask is shared by the two halves that the rotation exchanges, and neighbouring tiles must agree about the point they share. That is an exact cover problem with colours and multiplicities, which our `MCC` engine solves. Only afterwards does a second pass decide how to join the points inside each tile, so that every piece of the set is used once and the arcs close into one loop.

```
package main
import (
    "context"
    "flag"
    "fmt"
    "math/rand"
    "sort"
    "strings"
    "time"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Declarations 7 ⟩
⟨ Functions 5 ⟩
func main( ) {
    ⟨ Read the command line 4 ⟩
    ⟨ Take a census of the pieces 10 ⟩
    if *mode ≡ "census" {
        ⟨ Report the census and stop 12 ⟩
    }
    ⟨ Write the factored problem 16 ⟩
    ⟨ Search for a tiling 29 ⟩
}
```

4. The board and the piece set are flags, and so is every reduction. Chunking deserves a word: the blank tile is unique, so a solution places exactly one of it, and pinning it to a chosen symmetry orbit splits the search into independent pieces that can be run separately and stopped without loss.

⟨ Read the command line 4 ⟩ ≡

```
mode := flag.String("mode", "solve", "solve, census, or blanks")
rows := flag.Int("rows", 8, "rows")
cols := flag.Int("cols", 8, "columns")
set := flag.String("set", "nc32", "which pieces: 36, 48, or nc32")
loop := flag.Bool("loop", false, "also join the arcs into a single loop")
count := flag.Bool("count", false, "count every solution instead of stopping")
cross := flag.Int("cross", -1, "allow at most this many self-crossing pieces")
xclass := flag.Int("xclass", -1, "confine the crossing pieces to class pk")
symBlank := flag.Bool("symblank", false, "keep one blank placement per orbit")
blankAt := flag.Int("blankat", -1, "keep only orbit representative number k")
shuffle := flag.Int64("shuffle", 0, "permute the options with this seed")
hor := flag.Int("h", -1, "require exactly this many horizontal dominoes")
ver := flag.Int("v", -1, "require exactly this many vertical dominoes")
evenv := flag.Bool("evenv", false, "drop vertical placements at odd height")
limit := flag.Duration("limit", 0, "time limit, 0 for none")
flag.Parse()
R, C = *rows, *cols
```

This code is used in section 3.

5. The pieces. Number the six attachment points 0 through 5 in cyclic order around the tile. The 180° rotation is then the shift $k \mapsto k+3$, so a mask and its shift name the same on/off pattern, and I keep the smaller of the two.

⟨ Functions 5 ⟩ ≡

```
func rot(m int) int { return ((m ≪ 3) | (m ≫ 3)) & 63 }
func canon(m int) int {
    if r := rot(m); r < m {
        return r
    }
    return m
}
func pop(x int)(n int) {
    for; x ≠ 0; x &= x − 1 {
        n++
    }
    return
}
```

This code is also defined in sections 6, 8, 9, 14, 15, 28, 36, 37, 40.

This code is used in section 3.

6. A tile is a perfect matching of the points that are switched on. Pairing the first with each of the others in turn and recursing gives them all: one matching when nothing is on, fifteen when four points are on, fifteen again when all six are.

⟨ Functions 5 ⟩ +≡

```
func matchings(pts []int) [][]pair {
    if len(pts) ≡ 0 {
        return [][]pair{{}}
    }
    a, rest := pts[0], pts[1:]
    var out [][]pair
    for k, b := range rest {
        var others []int
        others = append(others, rest[:k]...)
        others = append(others, rest[k + 1:]...)
        for _, m := range matchings(others) {
            out = append(out, append([]pair{{a, b}}, m...))
        }
    }
    return out
}
```

7. Two chords of a circle cross when their endpoints interleave. That one line is the whole of exercise 152's "no crossings" condition.

⟨ Declarations 7 ⟩ ≡

```
type pair [2]int
```

This code is also defined in sections 13, 31, 35.

This code is used in section 3.

8. ⟨ Functions 5 ⟩ +≡

```
func crosses(p, q pair) bool {
    a, b := p[0], p[1]
    c, d := q[0], q[1]
    if a > b {
        a, b = b, a
    }
    if c > d {
        c, d = d, c
    }
    return (a < c ∧ c < b ∧ b < d) ∨ (c < a ∧ a < d ∧ d < b)
}
func noncrossing(m []pair) bool {
    for i := range m {
        for j := i + 1; j < len(m); j++ {
            if crosses(m[i], m[j]) {
                return false
            }
        }
    }
    return true
}
```

9. Since a tile and its half-turn are the same piece, a matching needs a name that both give. Sorting the chords of each of the two readings into a number and keeping the smaller does it.

⟨ Functions 5 ⟩ +≡

```
func mkey(m []pair) int {
    enc := func(sh int) int {
        ps := make([]int, len(m))
        for k, p := range m {
            a, b := (p[0] + sh) % 6, (p[1] + sh) % 6
            if a > b {
                a, b = b, a
            }
            ps[k] = a * 6 + b
        }
        sort.Ints(ps)
        v := 1
        for _, p := range ps {
            v = v * 36 + p
        }
        return v
    }
    a, b := enc(0), enc(3)
    if b < a {
        return b
    }
    return a
}
```

10. Now the census. Running over all sixty-four masks and all matchings of each, counting distinct pieces per pattern class, produces the multiplicities the answers prescribe—and produces them from the definition, not from the book. Set `36` keeps the two- and four-point pieces of exercise 151, set `48` keeps everything, and set `nc32` keeps only the pieces that never cross.

⟨ Take a census of the pieces 10 ⟩ ≡

```
seen := map[int]bool{}
mult := map[int]int{}
bySize := map[int]int{}
for mask := 0; mask < 64; mask++ {
    var on []int
    for b := 0; b < 6; b++ {
        if mask & (1 << b) != 0 {
            on = append(on, b)
        }
    }
    for _, m := range matchings(on) {
        ⟨ Skip this piece if the chosen set excludes it 11 ⟩
        k := mkey(m)
        if seen[k] {
            continue
        }
        seen[k] = true
        mult[canon(mask)]++
        bySize[pop(mask)/2]++
    }
}
total := 0
for _, v := range mult {
    total += v
}
```

This code is used in section 3.

11. ⟨ Skip this piece if the chosen set excludes it 11 ⟩ ≡

```
switch *set {
case "36":
    if pop(mask) ≠ 2 ∧ pop(mask) ≠ 4 {
        continue
    }
case "48":
case "nc32":
    if ¬noncrossing(m) {
        continue
    }
default:
    panic("unknown␣set␣" + *set)
}
```

This code is used in section 10.

12. ⟨ Report the census and stop 12 ⟩ ≡

```
fmt.Printf("set␣%s:␣%d␣on/off␣pattern␣classes,␣%d␣pieces␣in␣all\n",
    *set, len(mult), total)
for s := 0; s ≤ 3; s++ {
    fmt.Printf("␣␣␣%d␣subpath(s):␣%2d␣pieces\n", s, bySize[s])
}
return
```

This code is used in section 3.

13. The board. An attachment point of the array is the midpoint of a unit edge between two cells, or on the outer border. Horizontal edges get names `h`$i.j$ and vertical ones `v`$i.j$; a point is *internal* when it has a cell on both sides, and only internal points may be switched on, since a curve must not run off the board.

⟨ Declarations 7 ⟩ +≡

```
var R, C int   // rows and columns of the array
type att struct {
    name string
    internal bool
}
```

14. ⟨ Functions 5 ⟩ +≡

```
func hName(i, j int) string { return fmt.Sprintf("h%d.%d", i, j) }
func vName(i, j int) string { return fmt.Sprintf("v%d.%d", i, j) }
func hIn(i, j int) bool { return i ≥ 1 ∧ i ≤ R − 1 }
func vIn(i, j int) bool { return j ≥ 1 ∧ j ≤ C − 1 }
```

15. Here is the cyclic order, which everything else depends on. A horizontal domino at (i, j) is read left, top-left, top-right, right, bottom-right, bottom-left; a vertical one at (i, j) is read top, right-top, right-bottom, bottom, left-bottom, left-top. Both readings go once round the tile, so in both the half-turn is the shift by three.

⟨ Functions 5 ⟩ +≡

```
func points(i, j int, horiz bool) [6]att {
    if horiz {
        return [6]att{
            {vName(i, j), vIn(i, j)}, {hName(i, j), hIn(i, j)},
            {hName(i, j + 1), hIn(i, j + 1)}, {vName(i, j + 2), vIn(i, j + 2)},
            {hName(i + 1, j + 1), hIn(i + 1, j + 1)}, {hName(i + 1, j), hIn(i + 1, j)},
        }
    }
    return [6]att{
        {hName(i, j), hIn(i, j)}, {vName(i, j + 1), vIn(i, j + 1)},
        {vName(i + 1, j + 1), vIn(i + 1, j + 1)}, {hName(i + 2, j), hIn(i + 2, j)},
        {vName(i + 1, j), vIn(i + 1, j)}, {vName(i, j), vIn(i, j)},
    }
}
```

16. The factored problem. The items are of three kinds. Each on/off pattern class is a primary item whose multiplicity is how many pieces carry it, so that all of them get used. Each cell is a primary item of multiplicity one, so that the array is covered. Each attachment point is a secondary item whose colour is 0 or 1, so that two neighbouring tiles agree about whether the curve crosses the edge between them.

```
⟨ Write the factored problem 16 ⟩ ≡
    masks := make([]int, 0, len(mult))
    for m := range mult {
        masks = append(masks, m)
    }
    sort.Ints(masks)
    label, index := map[int]string{}, map[int]int{}
    for k, m := range masks {
        label[m], index[m] = fmt.Sprintf("p%d", k), k
    }
    var sb strings.Builder
    ⟨ Decide which blank placements to keep 27 ⟩
    ⟨ Write the item line 17 ⟩
    ⟨ Write one option per placement 21 ⟩
    input := sb.String()
```

This code is used in section 3.

17.

```
⟨ Write the item line 17 ⟩ ≡
    ⟨ Name the pattern items, with their multiplicities 18 ⟩
    for i := 0; i < R; i++ {
        for j := 0; j < C; j++ {
            fmt.Fprintf(&sb, "c%d.%d ", i, j)
        }
    }
    sb.WriteString("|")
    nh, nv := 0, 0
    for i := 1; i ≤ R − 1; i++ {
        for j := 0; j < C; j++ {
            fmt.Fprintf(&sb, " %s", hName(i, j))
            nh++
        }
    }
    for i := 0; i < R; i++ {
        for j := 1; j ≤ C − 1; j++ {
            fmt.Fprintf(&sb, " %s", vName(i, j))
            nv++
        }
    }
    sb.WriteString("\n")
    fmt.Printf("board %dx%d: %d cells, %d h + %d v = %d secondary items\n",
        R, C, R * C, nh, nv, nh + nv)
```

This code is used in section 16.

18. Ordinarily a pattern class is one item asking for exactly as many pieces as carry it. The `-cross` flag splits it in two: item `f`k for the pieces of class k that do not cross, item `x`k for those that do, and one further item `X` of multiplicity k that every crossing placement must also cover. At `-cross 0` the board must be tiled without a single crossing, which is exercise 152's question; raising it one notch measures how far the answer is from yes.

The budget item exists only when the budget is positive. An item of multiplicity zero is not something the engine will take—and it would have nothing to do anyway, since at `-cross 0` the crossing placements are simply never offered. I learned this the hard way: the first version asked for item `X` with bounds 0:0, and got a panic out of *Dance*.

The flag `-xclass` k narrows the budget further, to the crossing pieces of class `p`k alone. Ten of the twenty classes own one; running the flag over each of them in turn asks whether the deficiency has a culprit, and finds that it does not.

⟨ Name the pattern items, with their multiplicities 18 ⟩ ≡

```
ncross := map[int]int{}
if *cross ≥ 0 {
    ⟨ Count how many pieces of each class cross 20 ⟩
}
for _, m := range masks {
    if *cross < 0 {
        fmt.Fprintf(&sb, "%d|%s␣", mult[m], label[m])
        continue
    }
    fmt.Fprintf(&sb, "0:%d|f%s␣", mult[m] − ncross[m], label[m])
    if *cross > 0 ∧ ncross[m] > 0 ∧ (*xclass < 0 ∨ *xclass ≡ index[m]) {
        fmt.Fprintf(&sb, "0:%d|x%s␣", ncross[m], label[m])
    }
}
if *cross > 0 {
    fmt.Fprintf(&sb, "0:%d|X␣", *cross)
}
⟨ Name the H and V items of exercise 151 19 ⟩
```

This code is used in section 17.

19. Exercise 151 also fixes how many dominoes lie each way. Its answer introduces "two special items H and V" for that, with the multiplicities the part being solved requires: 18 and 18 for part (a), 32 and 4 for part (b). Answer 151 adds a parenthetical, that vertical placements at odd height may be dropped when $V = 4$; the flag `-evenv` does so, and the notes check that the shortcut loses nothing.

⟨ Name the H and V items of exercise 151 19 ⟩ ≡

```
if *hor ≥ 0 {
    fmt.Fprintf(&sb, "%d|H␣", *hor)
}
if *ver ≥ 0 {
    fmt.Fprintf(&sb, "%d|V␣", *ver)
}
```

This code is used in section 18.

20. ⟨ Count how many pieces of each class cross 20 ⟩ ≡

```
done := map[int]bool{}
for mask := 0; mask < 64; mask++ {
    var on []int
    for b := 0; b < 6; b++ {
        if mask & (1 << b) != 0 {
            on = append(on, b)
        }
    }
    for _, m := range matchings(on) {
        if noncrossing(m) {
            continue
        }
        k := mkey(m)
        if done[k] {
            continue
        }
        done[k] = true
        ncross[canon(mask)]++
    }
}
```

This code is used in section 18.

21. A placement is a domino at a cell together with an on/off mask. It is legal when every switched-on point of the mask is internal, and its option names the pattern class, the two cells, and the colour of each internal point.

⟨ Write one option per placement 21 ⟩ ≡

```
var opts []string
var ob strings.Builder
nrep := 0
emit := func(i, j int, horiz bool) {
    pts := points(i, j, horiz)
    ⟨ Note whether this placement represents its symmetry orbit 25 ⟩
    for mask := 0; mask < 64; mask++ {
        if mult[canon(mask)] == 0 {
            continue
        }
        ⟨ Skip the blank unless this chunk owns it 26 ⟩
        ⟨ Skip masks that would run off the board 22 ⟩
        ⟨ Append the option, once or twice 23 ⟩
    }
}
for i := 0; i < R; i++ {
    for j := 0; j+1 < C; j++ {
        emit(i, j, true)
    }
}
for i := 0; i+1 < R; i++ {
    if *evenv && i%2 == 1 {
        continue
    }
    for j := 0; j < C; j++ {
        emit(i, j, false)
    }
}
⟨ Shuffle the options if asked, then write them out 24 ⟩
```

This code is used in section 16.

22. ⟨ Skip masks that would run off the board 22 ⟩ ≡

```
ok := true
for b := 0; b < 6; b++ {
    if mask&(1<<b) != 0 && !pts[b].internal {
        ok = false
        break
    }
}
if !ok {
    continue
}
```

This code is used in section 21.

23. ⟨ Append the option, once or twice 23 ⟩ ≡

```
ob.Reset()
if *cross < 0 {
    fmt.Fprintf(&ob, "%s␣", label[canon(mask)])
} else {
    fmt.Fprintf(&ob, "f%s␣", label[canon(mask)])
}
fmt.Fprintf(&ob, "c%d.%d␣", i, j)
if horiz {
    fmt.Fprintf(&ob, "c%d.%d", i, j + 1)
} else {
    fmt.Fprintf(&ob, "c%d.%d", i + 1, j)
}
for b := 0; b < 6; b++ {
    if ¬pts[b].internal {
        continue
    }
    c := 0
    if mask & (1 ≪ b) ≠ 0 {
        c = 1
    }
    fmt.Fprintf(&ob, "␣%s:%d", pts[b].name, c)
}
if horiz ∧ *hor ≥ 0 {
    ob.WriteString("␣H")
} else if ¬horiz ∧ *ver ≥ 0 {
    ob.WriteString("␣V")
}
opts = append(opts, ob.String())
if *cross > 0 ∧ ncross[canon(mask)] > 0 ∧
    (*xclass < 0 ∨ *xclass ≡ index[canon(mask)]) {
    opts = append(opts, "x" + ob.String()[1:] + "␣X")
}
```

This code is used in section 21.

24. Shuffling matters more than it looks. The engine takes the options in the order given, and on a problem this size the order decides whether a solution turns up in minutes or not at all; running several orders in parallel is how the 8×12 array was eventually caught.

⟨ Shuffle the options if asked, then write them out 24 ⟩ ≡

```
if *shuffle ≠ 0 {
    rng := rand.New(rand.NewSource(*shuffle))
    rng.Shuffle(len(opts), func(a, b int) { opts[a], opts[b] = opts[b], opts[a] })
}
for _, o := range opts {
    sb.WriteString(o)
    sb.WriteString("\n")
}
```

This code is used in section 21.

25. The blank tile is unique, so every solution places exactly one of it, and the board and the piece set are both invariant under the symmetries of a rectangle. A solution may therefore be turned until its blank lies in a chosen fundamental domain. Keeping one placement per orbit makes the search cheaper without losing a solution; keeping *one* orbit makes an independent chunk of it.

⟨ Note whether this placement represents its symmetry orbit 25 ⟩ ≡

```
myBlank := −1
if blankRep(i, j, horiz) {
    myBlank = nrep
    nrep++
}
```

This code is used in section 21.

26. ⟨ Skip the blank unless this chunk owns it 26 ⟩ ≡

```
if mask ≡ 0 {
    if myBlank < 0 {
        continue
    }
    if *blankAt ≥ 0 ∧ myBlank ≠ *blankAt {
        continue
    }
}
```

This code is used in section 21.

27. A rectangle admits the four-group; a square admits all eight symmetries. A placement represents its orbit when no image of it is lexicographically smaller.

⟨ Decide which blank placements to keep 27 ⟩ ≡

```
blankRep := func(i, j int, horiz bool) bool {
    if ¬*symBlank ∧ *blankAt < 0 {
        return true
    }
    me := normCells(i, j, horiz)
    for _, f := range symmetries() {
        if less(image(i, j, horiz, f), me) {
            return false
        }
    }
    return true
}
```

This code is used in section 16.

28. ⟨ Functions 5 ⟩ +≡

```
func cellsOf(i, j int, horiz bool) [2][2]int {
    if horiz {
        return [2][2]int{{i, j}, {i, j + 1}}
    }
    return [2][2]int{{i, j}, {i + 1, j}}
}
func norm(cs [2][2]int) [4]int {
    a, b := cs[0], cs[1]
    if a[0] > b[0] ∨ (a[0] ≡ b[0] ∧ a[1] > b[1]) {
        a, b = b, a
    }
    return [4]int{a[0], a[1], b[0], b[1]}
}
func normCells(i, j int, horiz bool) [4]int { return norm(cellsOf(i, j, horiz)) }
func less(a, b [4]int) bool {
    for k := 0; k < 4; k++ {
        if a[k] ≠ b[k] {
            return a[k] < b[k]
        }
    }
    return false
}
func symmetries() []func(r, c int)(int, int) {
    lr, lc := R − 1, C − 1
    out := []func(r, c int)(int, int) {
        func(r, c int)(int, int) { return lr − r, lc − c },
        func(r, c int)(int, int) { return lr − r, c },
        func(r, c int)(int, int) { return r, lc − c },
    }
    if R ≡ C {
        out = append(out,
            func(r, c int)(int, int) { return c, lr − r },
            func(r, c int)(int, int) { return lc − c, r },
            func(r, c int)(int, int) { return c, r },
            func(r, c int)(int, int) { return lc − c, lr − r },
        )
    }
    return out
}
func image(i, j int, horiz bool, f func(r, c int)(int, int)) [4]int {
    cs := cellsOf(i, j, horiz)
    var im [2][2]int
    im[0][0], im[0][1] = f(cs[0][0], cs[0][1])
    im[1][0], im[1][1] = f(cs[1][0], cs[1][1])
    return norm(im)
}
```

29. Searching. The factored problem goes to the `MCC` engine, which handles the multiplicities. Each solution it returns is a tiling with an on/off pattern decided at every internal edge; whether the arcs can then be joined into one loop is a separate question, asked only when `-loop` is given.

⟨ Search for a tiling 29 ⟩ ≡

```
fmt.Printf("%d␣options,␣%d␣bytes", len(opts), len(input))
if *blankAt ≥ 0 {
    fmt.Printf(",␣blank␣orbits␣%d,␣chunk␣%d", nrep, *blankAt)
}
fmt.Println()
if *mode ≡ "blanks" {
    return
}
ctx := context.Background()
if *limit > 0 {
    var cancel context.CancelFunc
    ctx, cancel = context.WithTimeout(ctx, *limit)
    defer cancel()
}
mc := cells.NewMCC().WithContext(ctx)
mc.PulseInterval = 60 * time.Second
start := time.Now()
n, found := 0, false
res := mc.Dance(strings.NewReader(input))
go func() {
    for range res.Heartbeat {
        fmt.Printf("␣␣[%v]␣%d␣solutions,␣%d␣nodes\n",
            time.Since(start).Round(time.Second), n, mc.Nodes())
    }
}()
for sol := range res.Solutions {
    n++
    ⟨ Decide what to do with this solution 30 ⟩
}
if *loop {
    fmt.Printf("factored␣solutions␣examined:␣%d,␣single␣loop␣found:␣%v\n",
        n, found)
}
fmt.Printf("RESULT:␣%d␣solutions,␣%d␣nodes,␣%v,␣exhausted=%v\n",
    n, mc.Nodes(), time.Since(start).Round(time.Millisecond), ctx.Err() ≡ Λ)
```

This code is used in section 3.

30. ⟨ Decide what to do with this solution 30 ⟩ ≡

```
if ¬∗loop {
    if ¬∗count {
        break
    }
    continue
}
⟨ Read the solution as dominoes 32 ⟩
if nnode ≠ narc {
    fmt.Printf("inconsistent: %d endpoints, %d arcs\n", nnode, narc)
    break
}
if pick, yes := assign(ds, narc, len(ds)); yes {
    fmt.Printf("SINGLE LOOP on factored solution #%d\n", n)
    fmt.Println(verify(ds, pick, narc, len(ds)))
    ⟨ Print the placement 43 ⟩
    found = true
    break
}
```

This code is used in section 29.

31. Joining the arcs. Each domino of the tiling knows which of its points are on; the pieces that could sit there are the matchings of those points. Choosing one piece per domino, so that every piece of the set is used exactly once and the arcs close into a single cycle, is a second backtracking search—the "(nontrivial) program" the answer to 151 mentions, whose structure has a lot in common with Algorithm X.

⟨ Declarations 7 ⟩ +≡

```
type choice struct {
    piece int
    arc [][2]int
    pairs [][2]int    // the matching, as attachment-point indices
}
type dom struct {
    pt [6]att
    mask int
    cand []choice
    i, j int
    horiz bool
}
```

32. Reading a solution back means parsing each option for its cell, its orientation, and the colours it assigned; the mask follows, and with it the candidate pieces. Endpoints are numbered as they are first seen, so that the arcs become edges of a graph on consecutive integers.

⟨ Read the solution as dominoes 32 ⟩ ≡

```
node := map[string]int{}
id := func(s string) int {
    if v, ok := node[s]; ok {
        return v
    }
    node[s] = len(node)
    return len(node) − 1
}
ds := make([]dom, 0, len(sol))
narc := 0
for _, opt := range sol {
    ⟨ Turn one option into a domino 33 ⟩
}
nnode := len(node)
```

This code is used in section 30.

33. ⟨ Turn one option into a domino 33 ⟩ ≡

```
var i, j, i2, j2 int
fmt.Sscanf(opt[1], "c%d.%d", &i, &j)
fmt.Sscanf(opt[2], "c%d.%d", &i2, &j2)
horiz := i ≡ i2
col := map[string]int{}
for _, t := range opt[3:] {
    k := strings.LastIndexByte(t, ':')
    if k < 0 {
        continue  // the item X of the crossing budget carries no colour
    }
    c := 0
    if t[k + 1] ≡ '1' {
        c = 1
    }
    col[t[:k]] = c
}
d := dom{pt: points(i, j, horiz), i: i, j: j, horiz: horiz}
for b := 0; b < 6; b++ {
    if d.pt[b].internal ∧ col[d.pt[b].name] ≡ 1 {
        d.mask |= 1 ≪ b
    }
}
⟨ List the pieces that could sit here 34 ⟩
ds = append(ds, d)
```

This code is used in section 32.

34. ⟨ List the pieces that could sit here 34 ⟩ ≡

```
var on []int
for b := 0; b < 6; b++ {
    if d.mask & (1 ≪ b) ≠ 0 {
        on = append(on, b)
    }
}
for _, m := range matchings(on) {
    if *set ≡ "nc32" ∧ ¬noncrossing(m) {
        continue
    }
    arcs := make([][2]int, len(m))
    prs := make([][2]int, len(m))
    for t, p := range m {
        arcs[t] = [2]int{id(d.pt[p[0]].name), id(d.pt[p[1]].name)}
        prs[t] = [2]int{p[0], p[1]}
    }
    d.cand = append(d.cand, choice{mkey(m), arcs, prs})
}
narc += len(on)/2
```

This code is used in section 33.

35. A union-find structure with rollback keeps track of which endpoints already lie on a common strand. Adding an arc between two endpoints of the same strand would close a cycle, which is fatal unless it is the very last arc—so the test is one line, and undoing it is another.

⟨ Declarations 7 ⟩ +≡

```
type dsu struct {
    p, sz []int
    undo []int
}
```

36. ⟨ Functions 5 ⟩ +≡

```
func (d *dsu) find(x int) int {
    for d.p[x] ≠ x {
        x = d.p[x]
    }
    return x
}
func (d *dsu) union(a, b int) bool {
    a, b = d.find(a), d.find(b)
    if a ≡ b {
        return false
    }
    if d.sz[a] < d.sz[b] {
        a, b = b, a
    }
    d.p[b] = a
    d.sz[a] += d.sz[b]
    d.undo = append(d.undo, b)
    return true
}
func (d *dsu) mark() int { return len(d.undo) }
func (d *dsu) rollback(m int) {
    for len(d.undo) > m {
        b := d.undo[len(d.undo) − 1]
        d.undo = d.undo[:len(d.undo) − 1]
        d.sz[d.p[b]] −= d.sz[b]
        d.p[b] = b
    }
}
```

37. The search itself takes the dominoes with fewest candidates first, which is the same instinct that makes Algorithm X choose the item with the shortest list.

The count *npieces* is how many dominoes the board holds, not how big the piece set is. The two agree for the boards of exercise 151 and for the 8×12 array—forty-eight pieces, forty-eight dominoes—and I wrote *total* here at first without noticing the difference. On the chessboard with the full set they part company: 32 dominoes drawn from 48 pieces. Asking for 48 distinct pieces in 32 slots is asking for the impossible, so *assign* could never return true there, and I lost the better part of a day to searches that had been made unsatisfiable before they began.

```
⟨ Functions 5 ⟩ +≡
  func assign(ds []dom, narc, npieces int)([]int, bool) {
      ⟨ Order the dominoes, fewest candidates first 38 ⟩
      d := &dsu{p: make([]int, narc), sz: make([]int, narc)}
      for i := range d.p {
          d.p[i], d.sz[i] = i, 1
      }
      used := map[int]bool{ }
      pick := make([]int, len(ds))
      placed := 0
      var rec func(k int) bool
      rec = func(k int) bool {
          if k ≡ len(ord) {
              return placed ≡ narc ∧ len(used) ≡ npieces
          }
          ⟨ Try every piece that could sit at this domino 39 ⟩
          return false
      }
      if rec(0) {
          return pick, true
      }
      return Λ, false
  }
```

38.

```
⟨ Order the dominoes, fewest candidates first 38 ⟩ ≡
  ord := make([]int, len(ds))
  for i := range ord {
      ord[i] = i
  }
  for a := 1; a < len(ord); a++ {
      for b := a; b > 0 ∧ len(ds[ord[b]].cand) < len(ds[ord[b − 1]].cand); b-- {
          ord[b], ord[b − 1] = ord[b − 1], ord[b]
      }
  }
```

This code is used in section 37.

39. ⟨ Try every piece that could sit at this domino 39 ⟩ ≡

```
w := ord[k]
for ci, c := range ds[w].cand {
    if used[c.piece] {
        continue
    }
    mk := d.mark()
    ok := true
    for _, a := range c.arc {
        placed++
        if ¬d.union(a[0], a[1]) ∧ placed ≠ narc {
            ok = false
        }
    }
    if ok {
        used[c.piece] = true
        pick[w] = ci
        if rec(k + 1) {
            return true
        }
        delete(used, c.piece)
    }
    placed -= len(c.arc)
    d.rollback(mk)
}
```

This code is used in section 37.

40. Nothing is taken on trust. An independent pass rebuilds the graph from the chosen pieces, checks that every piece is distinct, that the arc count is right, that every endpoint has degree two, and then walks the loop from one endpoint to see that a single circuit covers all of it.

```
⟨ Functions 5 ⟩ +≡
    func verify(ds []dom, pick []int, narc, npieces int) string {
        adj := map[int][]int{}
        keys := map[int]bool{}
        arcs := 0
        for k := range ds {
            c := ds[k].cand[pick[k]]
            if keys[c.piece] {
                return "FAIL: a piece is used twice"
            }
            keys[c.piece] = true
            for _, a := range c.arc {
                adj[a[0]] = append(adj[a[0]], a[1])
                adj[a[1]] = append(adj[a[1]], a[0])
                arcs++
            }
        }
        ⟨ Check the counts and the degrees 41 ⟩
        ⟨ Walk the loop 42 ⟩
        return fmt.Sprintf("VERIFIED: one closed loop through all %d arcs, " +
            "%d distinct pieces, every endpoint of degree 2", narc, npieces)
    }
```

41.

```
⟨ Check the counts and the degrees 41 ⟩ ≡
    if arcs ≠ narc {
        return fmt.Sprintf("FAIL: %d arcs, want %d", arcs, narc)
    }
    if len(keys) ≠ npieces {
        return fmt.Sprintf("FAIL: %d pieces, want %d", len(keys), npieces)
    }
    for v, ns := range adj {
        if len(ns) ≠ 2 {
            return fmt.Sprintf("FAIL: endpoint %d has degree %d", v, len(ns))
        }
    }
```

This code is used in section 40.

42. ⟨ Walk the loop 42 ⟩ ≡

```
prev, cur, n := -1, 0, 0
for {
    nxt := adj[cur][0]
    if nxt ≡ prev {
        nxt = adj[cur][1]
    }
    prev, cur = cur, nxt
    n++
    if cur ≡ 0 {
        break
    }
}
if n ≠ narc {
    return fmt.Sprintf("FAIL: loop covers %d of %d arcs", n, narc)
}
```

This code is used in section 40.

43. The placement is printed in a form that can be drawn: one line per domino giving its cell, its orientation, its on/off mask in octal, and the matching that was chosen inside it.

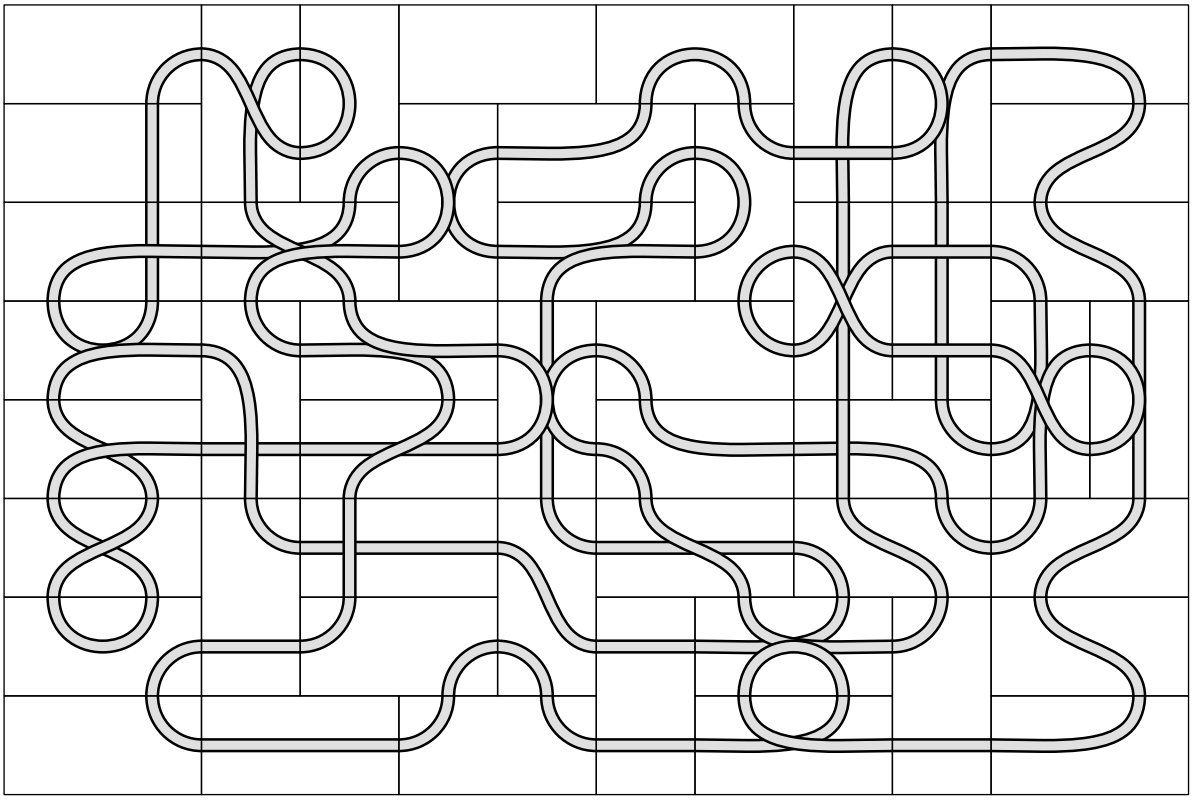
⟨ Print the placement 43 ⟩ ≡

```
fmt.Println("PLACEMENT")
for k, d := range ds {
    o := "v"
    if d.horiz {
        o = "h"
    }
    fmt.Printf("  %2d %s r%d c%d mask=%02o arcs=", k, o, d.i, d.j, d.mask)
    for _, p := range d.cand[pick[k]].pairs {
        fmt.Printf("%d-%d ", p[0], p[1])
    }
    fmt.Printf("key=%d\n", d.cand[pick[k]].piece)
}
```

This code is used in section 30.

44. The eight by twelve array. Exercise 152 asks for all forty-eight pieces in an 8×12 array. Its answer says only this about finding one: "One needs to be lucky to find a solution; the author struck it rich with Algorithm M after 35.1 Tμ." No arrangement is printed.

I looked for one by splitting the search. The blank piece is unique, so a solution places exactly one of it, and the 172 placements of a domino on an 8×12 board fall into 48 orbits under the four symmetries of a rectangle. Pinning the blank to one orbit gives an independent chunk, and the union of the forty-eight chunks is the whole problem. Six chunks ran in parallel; the fifth of them turned up a factored solution after 3.68 billion nodes, and the arcs inside it closed into a single loop at the first attempt. Node counts are not mems, so this is not a comparison with the author's figure—only a record of what it cost here.



All forty-eight path dominoes on an 8×12 array, their arcs forming one closed loop.
The one empty tile is the blank piece. Verified independently: 96 arcs, 48 distinct pieces, every endpoint of degree 2.

45. To reproduce the search, run the chunks separately:

```
verify -set 48 -rows 8 -cols 12 -mode blanks
verify -set 48 -rows 8 -cols 12 -blankat 4 -loop
```

The second line is the chunk that succeeded. It is deterministic, so it finds the same arrangement again; on this machine it took three hours.

46. Index.

Path Dominoes